

ElevanceSkills Internship Project Report

Build a Swiggy Food Delivery Backend System

Intern Name: PEDDIREDDY PRAJHANATHA

Internship Domain: Backend Development

Technologies: Node.js, Express.js, MongoDB, JWT Authentication

Table of Contents

1. Project Overview
2. Technologies Used
3. Task 1 - Fraud Detection and Order Validation System
4. Task 2 - Advanced Restaurant Search and Filtering System
5. Task 3 - Dynamic Surge Pricing for Delivery Fees
6. Task 4 - Smart Delivery Partner Assignment System
7. Task 5 - Real-Time Order Status and Notification System
8. Task 6 - Dynamic Restaurant Recommendation System
9. Important API Endpoints
10. Credentials for Testing
11. Practical Demonstration and API Testing Results
12. Conclusion

Project: Build a Food Swiggy Delivery Backend System

Internship Domain: Backend Development

Name: PEDDIREDDY PRAJHANATHA

Project Overview

This project is a backend food delivery system inspired by Swiggy. The system was enhanced with advanced backend features such as fraud detection, restaurant search and filtering, dynamic surge pricing, and smart delivery partner assignment using Node.js, Express.js, MongoDB, and JWT authentication.

Technologies Used

Node.js

Express.js

MongoDB

Mongoose

JWT Authentication

Postman

MongoDB Compass

Task 1: Fraud Detection and Order Validation System

- Implemented risk scoring for suspicious user activities.
- Monitored rapid order creation, repeated cancellations, coupon abuse, and refund abuse.
- Stored riskScore, isSuspicious, and fraudReason in orders.
- Added admin fraud management APIs for reviewing suspicious orders.

Task 2: Advanced Restaurant Search and Filtering System

- Implemented multi-filter restaurant search using MongoDB aggregation pipelines.
- Added filters for cuisine, rating, delivery time, vegetarian options, popularity, and price range.
- Implemented fuzzy text search for typo-tolerant restaurant searching.
- Added MongoDB indexes for optimized search performance.

Task 3: Dynamic Surge Pricing for Delivery Fees

- Implemented dynamic delivery fee calculation based on peak hours and demand.
- Added surge multiplier logic and active order monitoring by region.
- Stored deliveryFee and surgeMultiplier during order creation.
- Created admin APIs for configuring surge pricing settings.

Task 4: Smart Delivery Partner Assignment System

- Implemented automatic nearest delivery partner assignment using geo queries.
- Added delivery partner availability management and location tracking.
- Implemented reassignment flow when a delivery partner declines an order.
- Integrated active delivery workload balancing.

Task 5: Real-Time Order Status and Notification System

- Implemented real-time order lifecycle tracking with multiple status stages including Order Placed, Accepted, Preparing, Out for Delivery, Delivered, and Cancelled.
- Added secure order status update functionality for administrators and restaurant owners.
- Implemented notification and event simulation for instant order status updates using API responses and backend console logs.
- Enabled users to retrieve the latest order status dynamically through order tracking APIs.

Task 6: Dynamic Restaurant Recommendation System

- Implemented personalized restaurant recommendation system using MongoDB aggregation pipelines.
- Generated restaurant recommendations based on user order history, preferred cuisines, restaurant ratings, and popularity.
- Added dynamic recommendation scoring and fallback recommendations for users without previous order history.

Important API Endpoints

POST /api/orders/create

POST /api/orders/calculate-delivery-fee

GET /api/restaurants/search

POST /api/delivery/set-status

GET /api/orders/:orderId

GET /api/admin/fraud/orders

POST /api/admin/surge-settings

GET /api/orders/my

PUT /api/orders/update-status/:orderId

POST /api/auth/login

GET /api/restaurants/recommendations/:userId

Credentials for Testing

Admin

Email: admin@test.com

Password: admin123

User

Email: rahul@test.com

Password: password

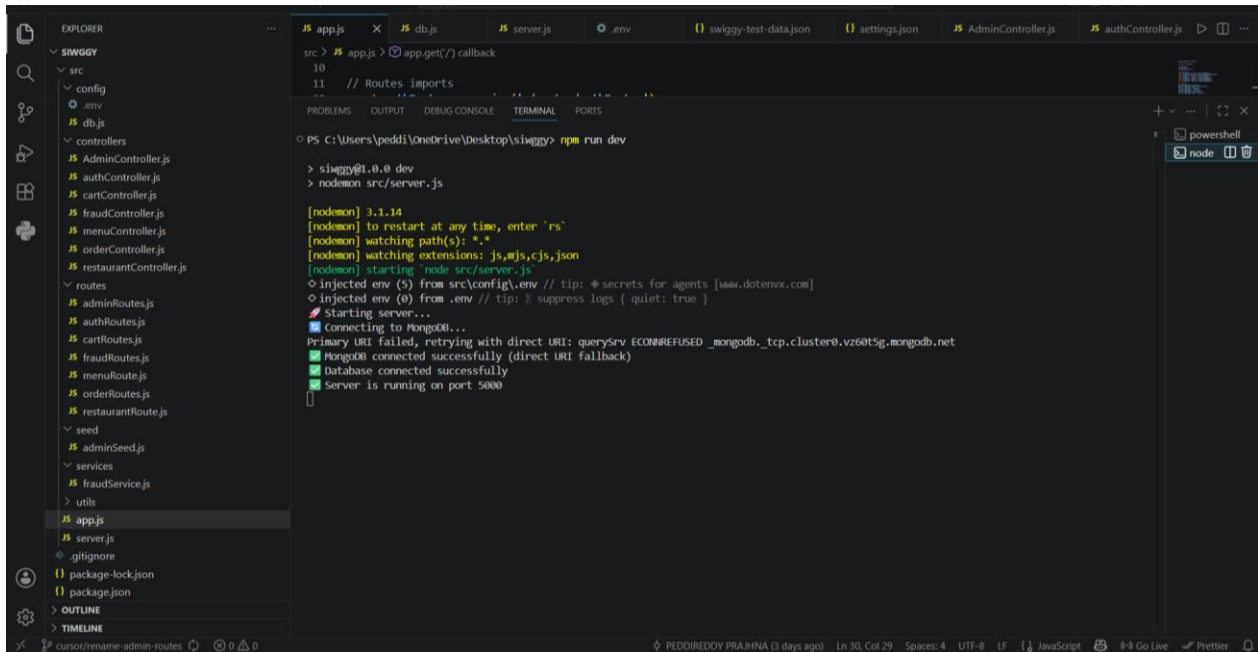
Delivery Partner

Email: delivery1@test.com

Password: password

Practical Demonstration and API Testing Results

Server Startup and MongoDB Connection Output :



The screenshot shows a VS Code editor with a project named 'SWIGGY'. The Explorer panel on the left shows the file structure, including 'src', 'config', 'env', 'db.js', 'controllers', 'routes', 'seed', 'services', 'utils', and 'app.js'. The main editor shows the 'app.js' file with the following code:

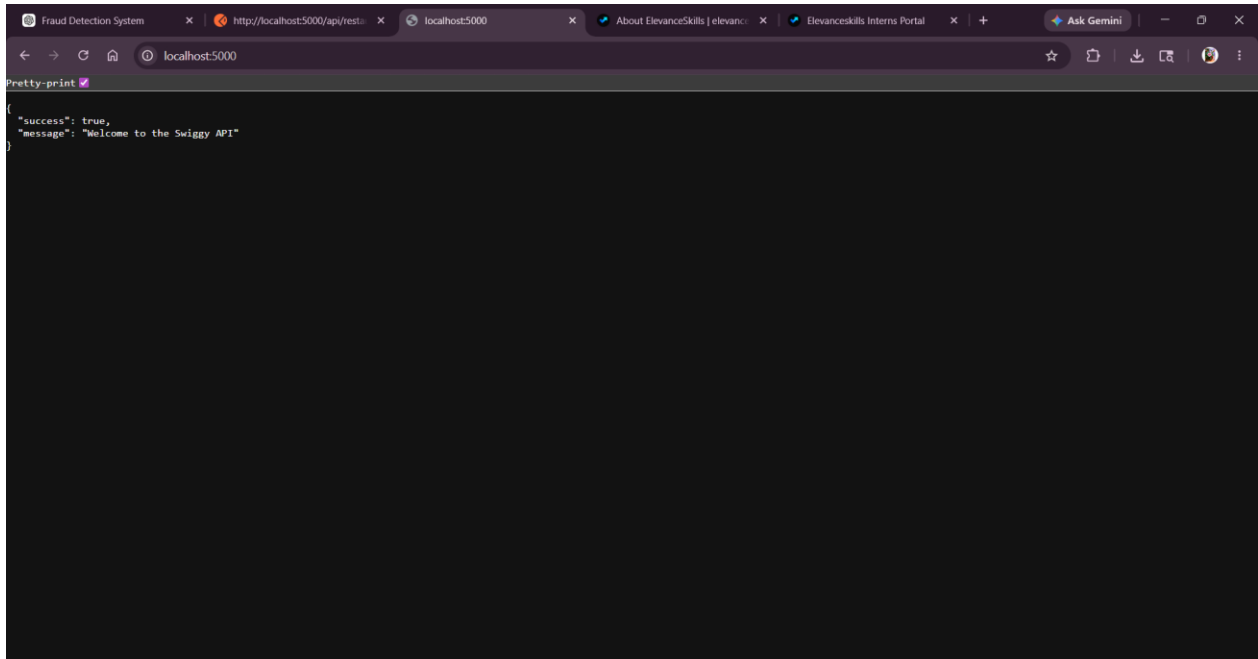
```
src > JS app.js > app.get('/') callback
10
11 // Routes imports
```

The terminal output shows the command 'npm run dev' being executed, which starts the server using 'nodemon src/server.js'. The output includes the following messages:

```
PS C:\Users\pedd\OneDrive\Desktop\swiggy> npm run dev
> swiggy@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
◊ injected env (5) from src/config/.env // tip: ✎ secrets for agents [www.dotenvx.com]
◊ injected env (0) from .env // tip: ✎ suppress logs { quiet: true }
Starting server...
Connecting to MongoDB...
Primary URI failed, retrying with direct URI: querysrv_ECONNREFUSED_mongodb_tcp.cluster0.v260t5g.mongodb.net
MongoDB connected successfully (direct URI fallback)
Database connected successfully
Server is running on port 5000
```

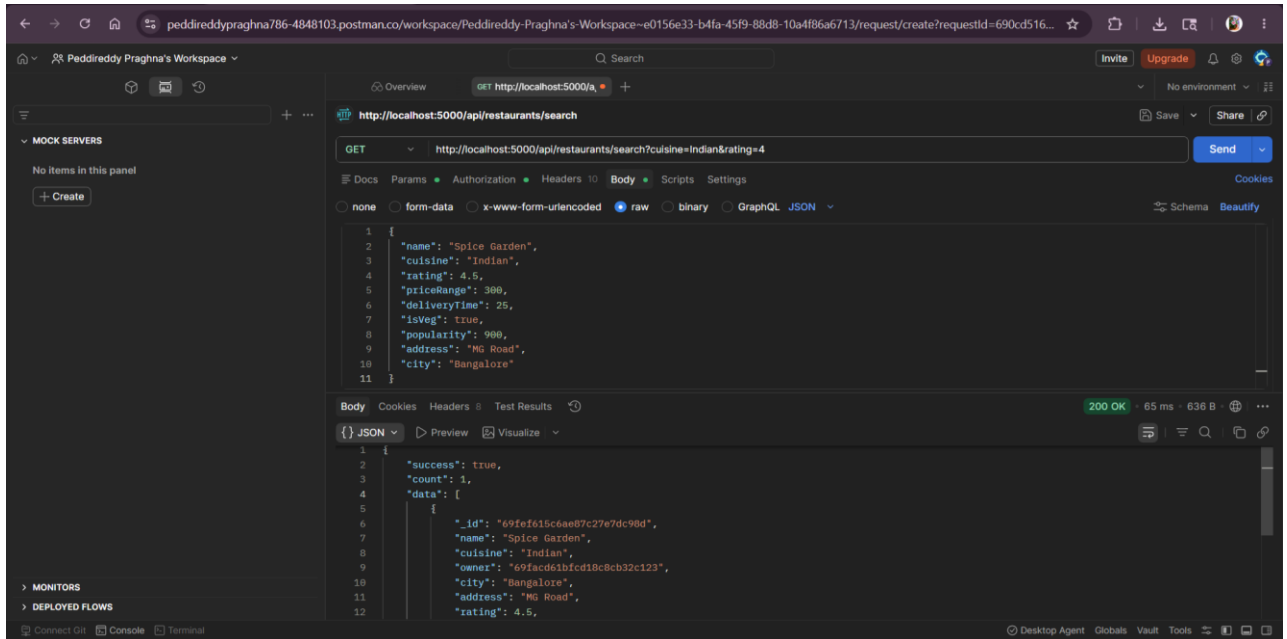
API Server Running Successfully on Localhost:5000



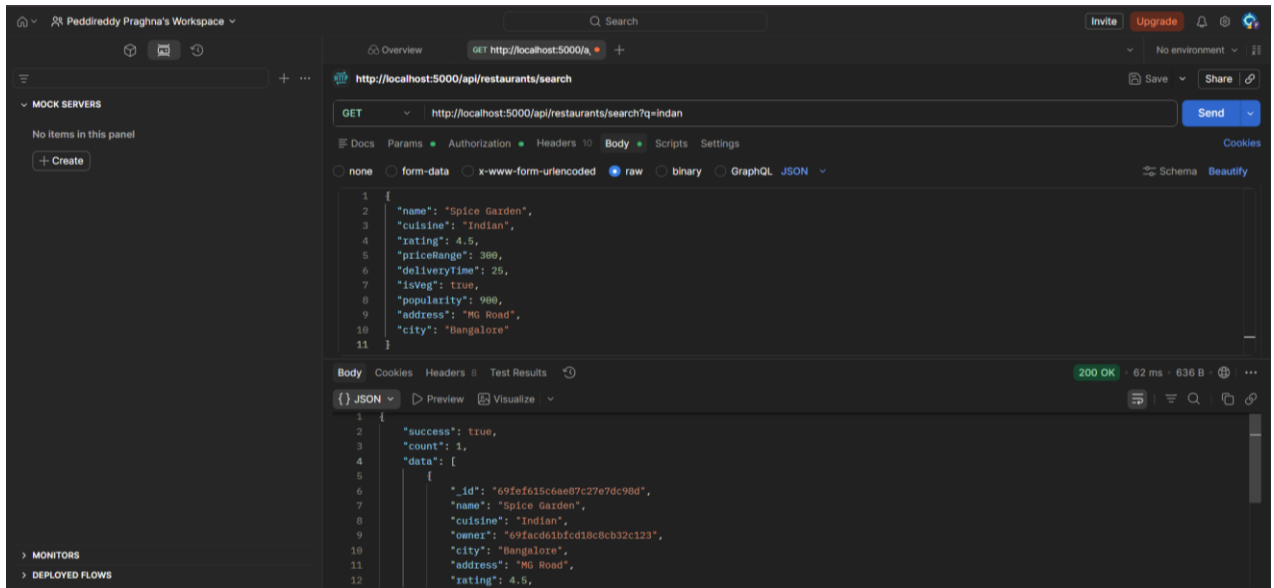
The screenshot shows a web browser with the address bar set to 'localhost:5000'. The page displays the JSON response from the API endpoint 'http://localhost:5000/api/rest/':

```
{
  "success": true,
  "message": "Welcome to the Swiggy API"
}
```

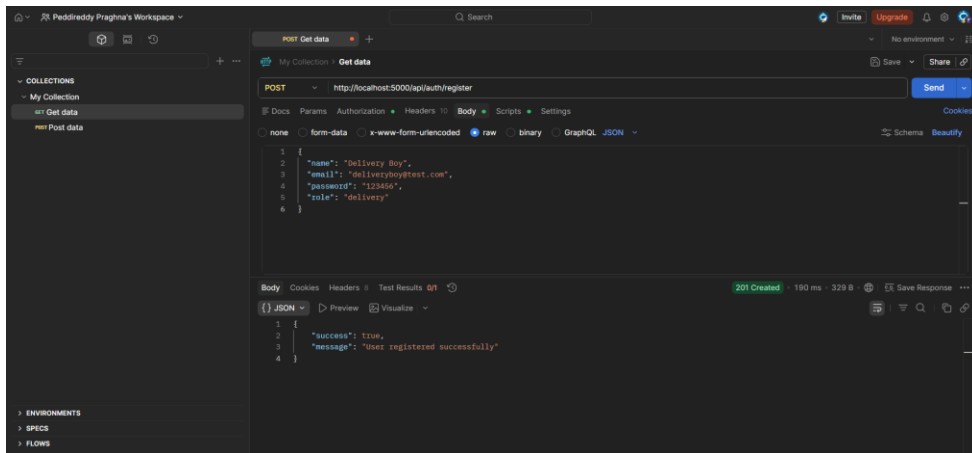
Restaurant Search Using Cuisine and Rating Filters:



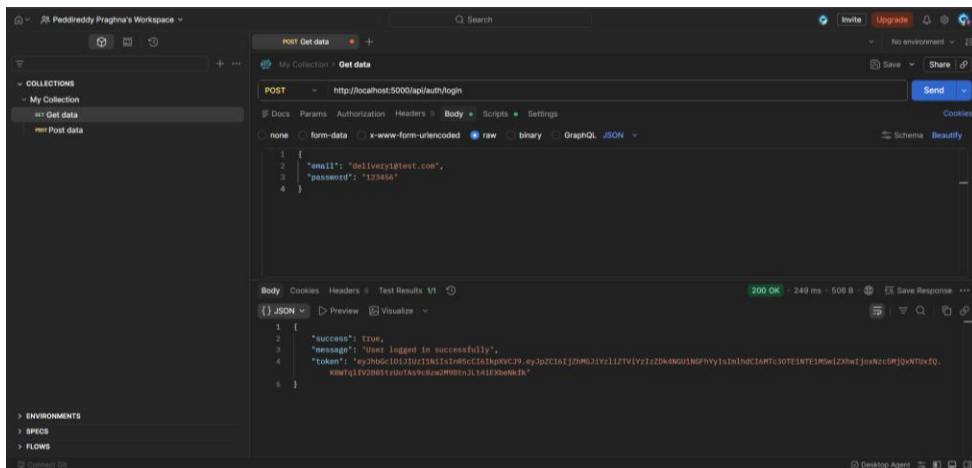
Fuzzy Text Search Functionality Verification:



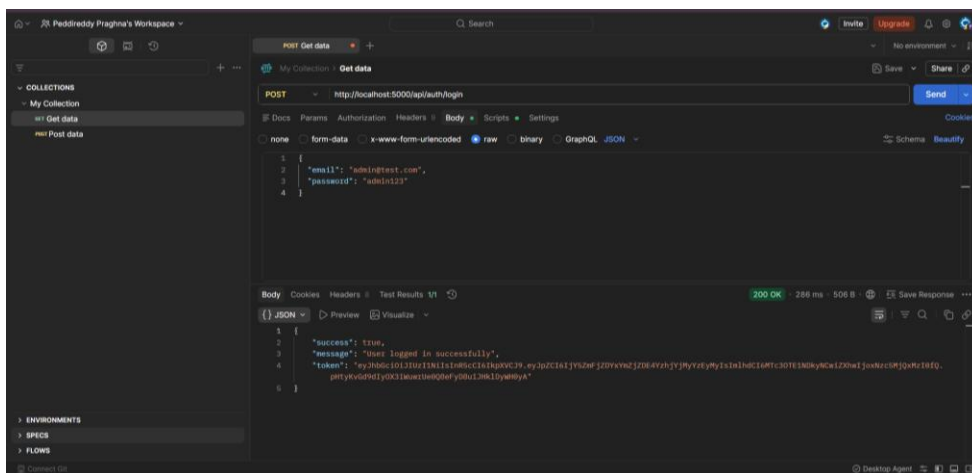
Delivery Partner Account Creation Successfully Verified:



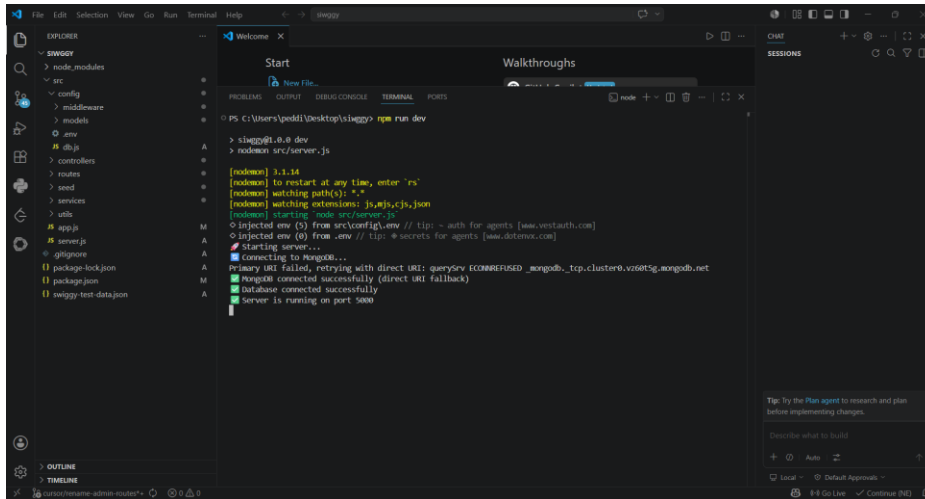
Delivery Partner Login Authentication Verification:



Administrator Login Authentication Verification:



Delivery Service Server Running Successfully:

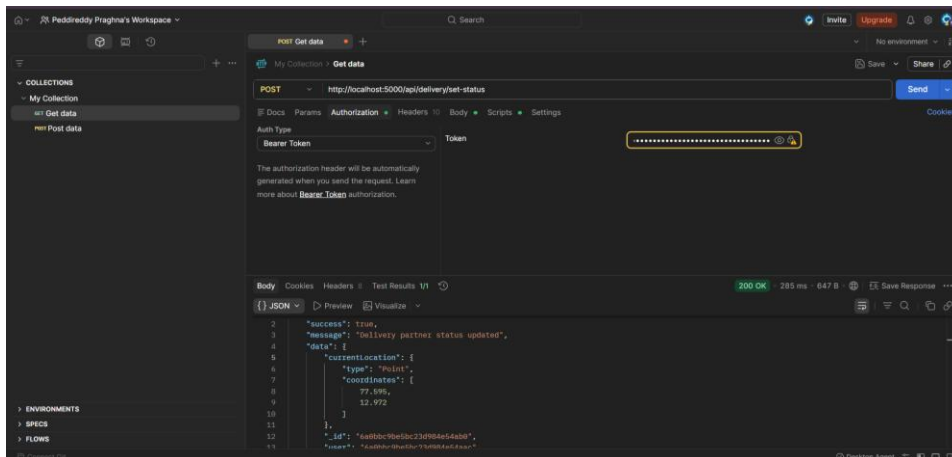


The screenshot shows a VS Code editor with a file explorer on the left displaying a project structure for 'siaggy'. The main editor area shows a terminal window with the following output:

```
Start
PS C:\Users\pedd\desktop\siaggy> npm run dev
> siaggy@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
O injected env (0) from src/config/env // tip: ~ auth for agents [www.vestauth.com]
O injected env (0) from .env // tip: ~ secrets for agents [www.demos.co]
A starting server...
A Connecting to MongoDB...
M Primary URI failed, retrying with direct URI: querySvc ECONNREFUSED _mongodb_.tcp.cluster0.v068t5g.mongodb.net
M MongoDB connected successfully (direct URI fallback)
M Database connected successfully
A Server is running on port 5000
```

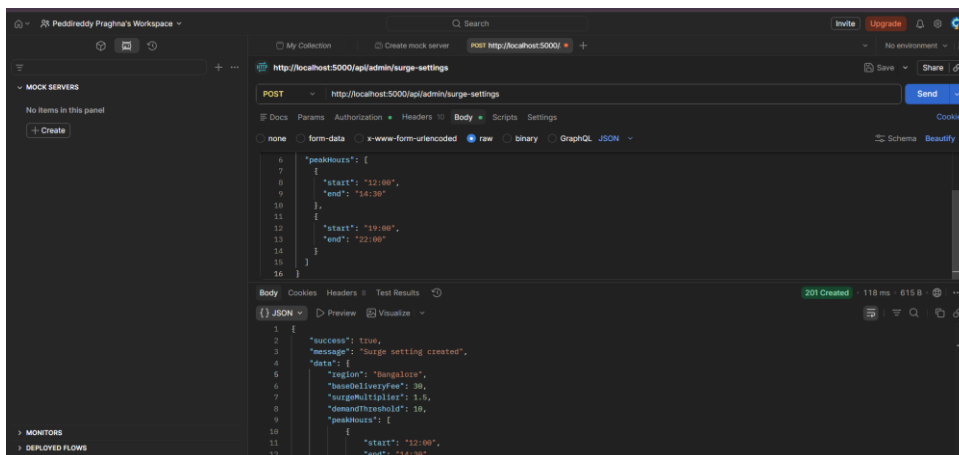
Delivery Partner Availability and Location Status Update:



The screenshot shows a Postman interface with a collection named 'My Collection' containing a request 'Get data'. The request is a POST to 'http://localhost:5000/api/delivery/part-status'. The 'Auth Type' is set to 'Bearer Token'. The response is a 200 OK status with a JSON body:

```
{
  "success": true,
  "message": "Delivery partner status updated.",
  "data": {
    "currentLocation": {
      "type": "Point",
      "coordinates": [
        77.599,
        12.972
      ]
    },
    "_id": "6a0b0c7e0bc33d90454a0b",
    "name": "Banshi Bhatia"
  }
}
```

Administrator Surge Pricing Settings Configuration Verification:



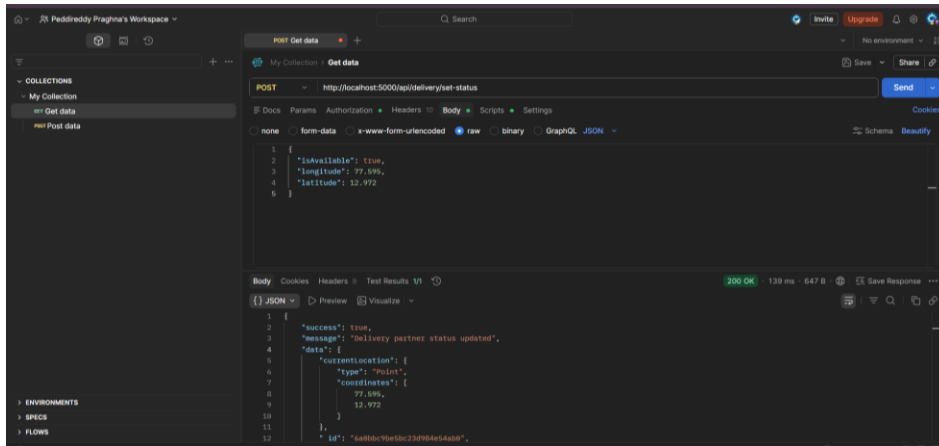
The screenshot shows a Postman interface with a collection named 'My Collection' containing a request 'admin/surge-settings'. The request is a POST to 'http://localhost:5000/api/admin/surge-settings'. The 'Body' is set to 'raw' and contains the following JSON:

```
{
  "peakHours": [
    {
      "start": "12:00",
      "end": "14:30"
    },
    {
      "start": "19:00",
      "end": "22:00"
    }
  ]
}
```

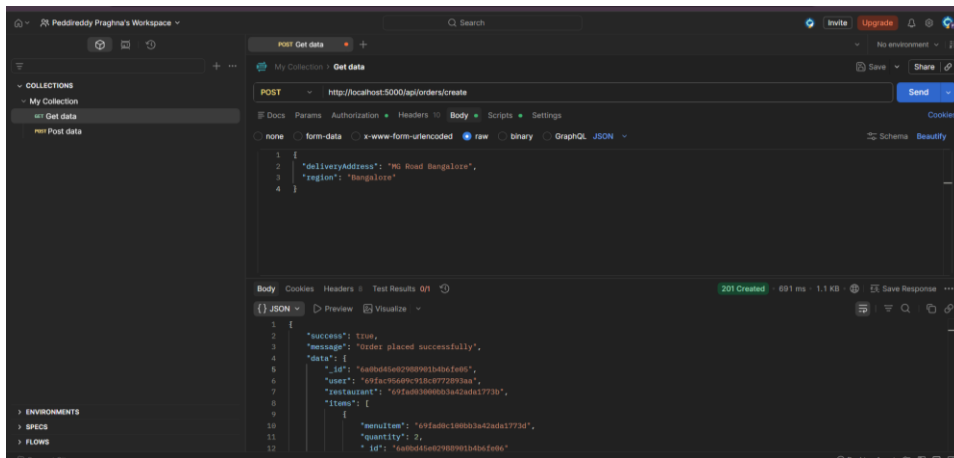
The response is a 201 Created status with a JSON body:

```
{
  "success": true,
  "message": "Surge setting created",
  "data": {
    "region": "Bangalore",
    "baseDeliveryFee": 30,
    "surgeMultiplier": 1.5,
    "demandThreshold": 10,
    "peakHours": [
      {
        "start": "12:00",
        "end": "14:30"
      }
    ]
  }
}
```

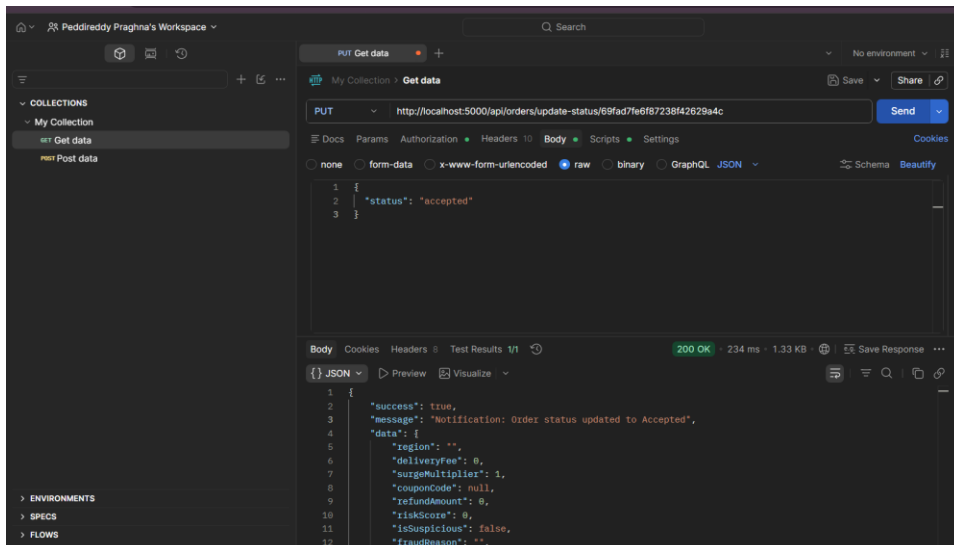
Delivery Partner Status and Location Update Verification:



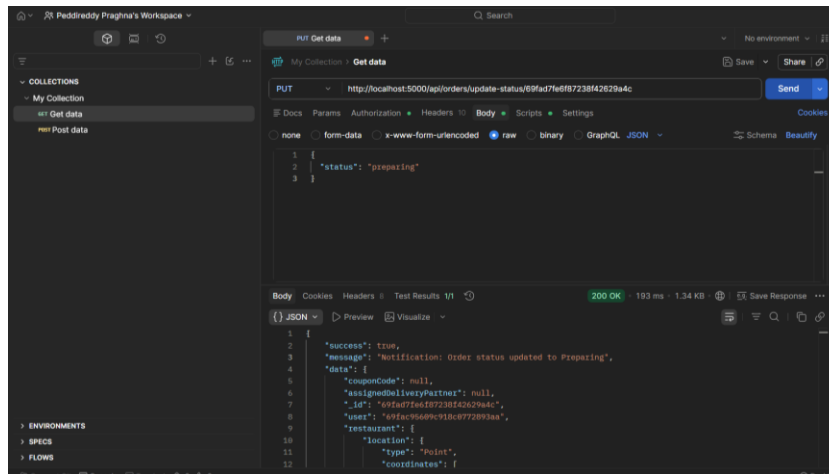
Order Creation and Automatic Delivery Partner Assignment Verification:



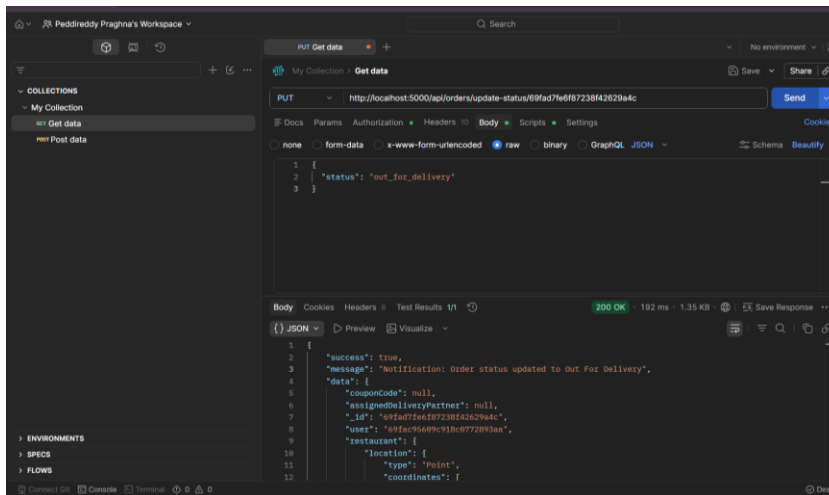
Order Status Updated to Accepted:



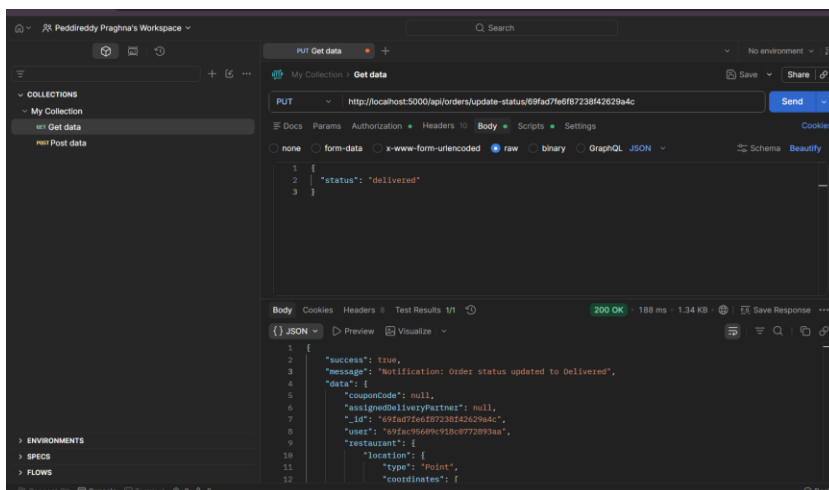
Order Status Updated to Preparing:



Order Status Updated to Out for Delivery:



Order Successfully Delivered:



Conclusion

Successfully implemented and tested all six required backend systems including Fraud Detection and Order Validation, Advanced Restaurant Search and Filtering, Dynamic Surge Pricing, Smart Delivery Partner Assignment, Real-Time Order Status Tracking and Notification System, and Dynamic Restaurant Recommendation System. All APIs were verified successfully using Postman, MongoDB Compass, and backend server testing. The project was integrated with MongoDB aggregation pipelines, JWT authentication, geo-based queries, dynamic pricing algorithms, recommendation scoring systems, and real-time order lifecycle management. Proper authorization, validation, error handling, and database indexing were implemented throughout the application to ensure scalability, security, and production-level performance. The complete backend system is fully functional, submission-ready, and successfully demonstrates all internship requirements with proper API execution and database operations.